Prevalidated Elimination for iPRA

A smaller complete checker, explicit axioms, and the exact limits of the improvement

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

This investigation compares all seventeen current draft archives in *ProveIt* at commit `76a2651601d17b34d46753f189ea026a21ce8d29`. The observed complete-checker minima are 558 normalized Python bytes, 262 significant tokens, and 296 abstract-syntax-tree nodes, attained by different programs. We obtain a 547-byte implementation with explicit list recognition, and a 539-byte implementation using the inherited compact list test. An auxiliary generic-traversal implementation lowers the token minimum by one and the AST minimum by four, subject to its stated malformed-input qualification. No single delivered program beats all three old minima.

The new checking discipline fixes an elimination motive and both of its instances before recursively proving the major premise. The successful major check is then combined with construction of the exported local assumption. Two internal failure sentinels remove redundant control checks. We prove soundness and canonical completeness; the changed raw JSON acceptance language is explicitly not claimed to coincide with the old one. Concrete examples witness incomparability in both directions.

Separately, a structural classification of standard positive-iteration terms yields a finite-tuple obstruction for operations with a two-sided unit and actual equality. An explicit quotient construction shows the essential limit of that obstruction.

The arithmetic and logical axiomatization is inherited, fully stated, and accompanied by its attributed internal arithmetic and logical reconstruction. It remains equiconsistent and bi-interpretable with full first-order intuitionistic primitive recursive arithmetic, with identity numerical composites on the whole domain. The result is a modest, audited checker reduction. A significant new simplest axiomatization has not been established. The proofs are written mathematical proofs, not Lean or Rocq formalizations.

Status. The byte frontier is improved, including by an explicit-list program. The substantial axiomatic breakthrough requested remains open. The minimum observed in a finite collection is not a proved lower bound on every possible presentation. Gzip measurements are informational.

Contents

1 Result and reproducible comparison **3**

2 The complete native presentation **4**

3 The new kernel and its correctness argument **7**

4 What is interpreted, and why both composites are identities **12**

5 Strict dependence in the positive-iteration term algebra **14**

6 Larger redesigns examined, and the remaining target **18**

A Attribution and the complete inherited mathematical bridge **20**

B Complementary introductions with one local assumption **20**

C Arithmetic strength and both ordinary interpretation composites **25**

D Internal scalar predecessor bootstrap **27**

E From literal-equation induction to full intuitionistic PRA **31**

F Pairing, primitive recursion, and internal numerical composites **36**

1 Result and reproducible comparison

The requested target was a substantially simpler axiomatization of intuitionistic primitive recursive arithmetic, below the current draft frontier when the whole proof checker is charged. The verified positive result here is narrower: a smaller complete implementation of the existing native presentation. Its numerical language, logical principles, and interpretation domain remain those of the inherited calculus. This distinction determines the claims throughout the report.

1.1 The actual current baseline

The pinned snapshot contains seventeen direct archives. A recursive source census, including nested historical archives, finds 3,557 Python occurrences and 443 distinct source byte strings. An independent normalizer agrees with the inherited measurement program on all 443. Thirty-eight distinct sources lie at or below at least one current complete-checker coordinate minimum; three are complete checkers and thirty-five are supporting utilities. A build script or measurement program that calls another checker is not counted as a complete checker. These three comprise the certified-zero byte and joint profiles and the affine-supply AST profile, which ties the AST minimum. The exact classification and source hashes are included.

Complete implementation	Bytes	Tokens	AST	Gzip-9
Old coordinate envelope	558	262	296	—
Old byte leader	558	264	310	345
Old structural leader	598	262	296	350
Old explicit-list version	566	264	308	351
New explicit-list version	547	263	307	347
New byte profile	539	263	309	342
New structural profile (qualified)	576	261	292	345

Raw and normalized lengths coincide for these compact files. The envelope is not the size of a single program.

The explicit-list implementation is the recommended small executable. It avoids the generic traversal's dependence on the runtime recursion limit for malformed strings. The byte profile replaces only the explicit list predicate by the inherited exact-JSON-type expression `a*0 == []`. The structural profile uses exception-based generic traversal, as its directly compared predecessor does. Its canonical proofs are complete in the ideal resource-unbounded reading; an arbitrary nonempty malformed string can diverge under an ideal unlimited stack. Actual CPython rejects such strings through a recursion exception and subsequent scalar type failure. This qualification is not attached to the explicit-list or byte profiles.

Every row for a program describes one complete source file. The envelope row combines minima from different programs. The new programs improve that envelope to 539 bytes, 261 tokens, and 292 AST nodes, also attained by different files. The 539-byte program does not establish the 261-token or 292-node records. In particular, this work does not produce one program strictly below all three old coordinates.

1.2 What is charged

The complete module is measured: JSON input, syntax opening and validation, arithmetic and logical rule dispatch, context management, fresh-name allocation, and the verdict. No producer, translator,

theorem database, or arithmetic evaluator is imported by a measured checker. Python and its standard JSON implementation are the common runtime, excluded by the same convention in every compared source.

Normalization removes comments and genuine docstrings, applies `ast.unparse`, and appends a newline. Significant-token counts exclude whitespace, comments, indentation, and end markers. AST counts include the entire parsed module. The recorded runtime is CPython 3.12.14 with zlib 1.3.2. All newly measured compact sources are saved in their normalized form. Each exact source is also supplied compressed by

```
gzip.compress(source_bytes, compresslevel=9, mtime=0)
```

with the ordinary gzip wrapper and no filename. Both raw and normalized gzip sizes are recorded. Gzip is an auxiliary figure, not an optimization target or an asserted mathematical complexity measure.

Acceptance means exit status zero and exactly `ACCEPT` on standard output. Ordinary checking or parsing exceptions reject with nonzero exit status. This convention and the output driver are unchanged from the leading current comparison. No saving is attributed to dropping a rejection wrapper or weakening the verdict protocol.

1.3 New contribution and inherited components

The new contribution is a sequencing theorem for the checker and a small implementation of that sequencing. An elimination's motive is validated before its major proof allocates internal names. The major proof is still checked. Its successful return is reused when constructing the next context entry. Internal non-JSON sentinels then remove a redundant truthiness guard and shorten missing-image failure handling. The ablation receipt distinguishes these changes from the inherited arithmetic and logical theory.

A separate mathematical contribution classifies the standard numerical functions generated by positive iteration as constants, projections, or functions with a strict growth property. It excludes every nontrivial term-defined operation with a two-sided unit on a finite-tuple domain when equality is actual equality. An explicit integer-difference quotient shows why this obstruction does not extend to arbitrary quotient equality. This is a new analysis relative to the inspected drafts, not a smaller axiomatization or a literature-wide priority claim.

The scalar iterator, paired closure binder, equation-only progress rule, reconstruction of ordinary intuitionistic logic, predecessor bootstrap, actual-equality reflection, pairing compiler, certified-zero computation restriction, arity representation, and generic-traversal profile all come from the supplied drafts. They are attributed explicitly. Their full bridge is reproduced because it is needed to substantiate bi-interpretability; its length is not evidence of novelty here.

An attained minimum among inspected programs is an upper bound on an unknown global optimum. It is not a proved lower bound on all theories, languages, domains, or implementations. This report improves an attained source record and does not claim absolute simplestness.

2 The complete native presentation

We use full first-order intuitionistic PRA as the target: fixed finite primitive-recursive function descriptions, their defining equations, ordinary intuitionistic predicate logic with equality, and quantifier-free induction. In the full PR language the usual atomic-induction presentation is equivalent; Salehi uses that formulation [10]. The smaller scalar language below requires an internal proof of that

equivalence, which is supplied in the inherited appendices.

2.1 Numerical syntax and computation

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h).$$

Only the occurrence of v in h is bound. Every other parameter is fixed during iteration. A fixed earlier-stage update description gives a first-order function symbol with counter, initializer, and passive parameters as arguments. There is no number variable ranging over function descriptions, and no universally total evaluator is assumed.

The two unrestricted computation schemas are

$$I(0, a; v.h) = a, \tag{I0}$$
$$I(Sn, a; v.h) = h[I(n, a; v.h)/v]. \tag{IS}$$

The measured checker directly supplies the following restricted instances: I0 when a is not the literal zero term, and IS when neither n nor a is literal zero. These are syntactic restrictions, not assertions that the terms have positive numerical values.

Lemma 2.1 (The inherited certified-zero repair)**.** *The restricted computations derive every instance of I0 and IS using equality transport. Each excluded computation leaf can be replaced by exactly three inference nodes.*

Proof. Let $1 = S0$, $2 = S1$, and $C = I(2, 1; v.0)$. Its direct IS instance gives $C = 0$: its predecessor is 1, its initializer is 1, and the update ignores the previous state. In a forbidden computation equation replace every offending literal-zero initializer or predecessor, or both, by one fresh variable w. Call the resulting equation template $F(w)$. The equation $F(C)$ is a permitted direct computation. Transport $C = 0$ through F to obtain $F(0)$, exactly the requested original equation. The two computation leaves and one transport are three nodes. The proof does not assume a zero-initializer computation to establish $C = 0$. □

2.2 Formula syntax

Native formulas are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The designated absurd formula is $\bot_* := (0 = S0)$. The following ordinary readings explain the interpretations but are not additional primitive logical syntax:

$$\mathsf{Q}_y B = B(0) \to \forall y\, B(y), \qquad \mathsf{D}_{x,y} B = \exists x \big(B(x, 0) \to \forall y\, B(x, y)\big).$$

All bound names are renamed before capture-free substitution. A context Γ is a finite reusable list of native formulas. The primitive rules are exactly those below, together with the restricted computation schemas just specified.

Hypothesis and equality transport.

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ (\mathrm{Tr}).$$

Here $F(w)$ is any fixed native formula template. There is no primitive reflexivity rule. For example, with $r = I(0, t; v.0)$, using the computation $r = t$ twice in transport with $F(w) := (w = t)$ derives $t = t$.

Closure and paired elimination.

$$\frac{\Gamma \vdash \mathsf{Q}_y B \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ (\mathrm{QE}), \qquad \frac{\Gamma \vdash \mathsf{D}_{x,y} B \quad \Gamma, \mathsf{Q}_y B(a, y) \vdash A}{\Gamma \vdash A}\ (\mathrm{DE}).$$

For DE the parameter a is fresh for the retained context, conclusion, and passive parameters of the major formula. The displayed local hypothesis is discharged.

The two paired introductions.

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}\ (\mathrm{DI}_{\mathrm{case}}),$$

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)}\ (\mathrm{DI}_{\mathrm{progress}}).$$

The progress rule is permitted only when E is a literal numerical equation. An equation underneath a quantifier or paired binder is not a literal equation. In both rules z is fresh for Γ, the witness t, and all passive parameters. The indicated local hypothesis is discharged. Case introduction applies to every native formula, including equations. Each introduction retains exactly one local assumption.

Structural conventions. Weakening, contraction, exchange, ordinary-assumption cut, renaming, and capture-free numerical substitution are admissible by induction on derivations. These are metatheorems, not an unlisted theorem oracle. If substitution turns a previously permitted computation initializer or predecessor into literal zero, Lemma 2.1 supplies the substituted computation instance. The root-constructor restriction on progress is preserved by numerical substitution. A concrete proof supplies all premises of every selected rule; no search procedure is part of the trusted kernel.

2.3 The implemented computation-supply rule

Instead of recognizing a computation leaf by comparing it to the goal, the implementation inserts a generated local hypothesis and continues. Its mathematical justification is the following admissible rule:

$$\frac{\vdash H(\vec{0}) \qquad \Gamma, H(\vec{b}) \vdash A}{\Gamma \vdash A}, \quad \vec{b} \cap \mathrm{FV}(\Gamma, A) = \varnothing. \tag{1}$$

Substitute zero simultaneously for the new names in the continuation, renaming later eigenparameters first, and cut its remaining assumption against the given theorem. Only the specific computation generators described in Section 3 are implemented. There is no general recognizer for the theorem premise of (1).

An open generated hypothesis need not hold at every new parameter value. For example an admitted raw packet can generate $I(0, 0; v.0) = b$, whose zero instance is a computation theorem but

whose instance at $b = 1$ is false. Validation of the retained goal before that new name is admitted is essential. The soundness argument is substitution and cut, not uniform truth of every supplied open equation.

2.4 Derived cases and induction

For a literal equation $E(y)$, introduce $\mathsf{D}_{u,y}E(y)$ with unused u by progress, eliminate it to obtain its closure, and use the proved base. This derives

$$\frac{\Gamma \vdash E(0) \quad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}.$$

The parameters remain fixed throughout. Likewise case introduction with unused witness, followed by DE and QE, derives numerical cases for every native A:

$$\frac{\Gamma \vdash A(0) \quad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}.$$

These derived rules must not be confused: the second gives cases, not induction on arbitrary quantified formulas. The primitive progress restriction is retained throughout the executable programs.

3 The new kernel and its correctness argument

The following is the entire recommended small checker. The explanatory `code/readable_checker.py` separately implements the same explicit-list discipline with named variables and comments; the small program does not import it.

```
import json
k = 0
i = lambda a, t=0j, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k <= a
    < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m in ([s, u], s and 1j or [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g) or m * 3 if m else [[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a :=
     i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == m) and k)])
q = json.load(open(0)).pop
c(q(), [])
print('ACCEPT')
```

3.1 Certificate representation

Input is one JSON array. The last item is the requested closed native formula. Earlier items are four-field records $[m, s, u, t]$, consumed backwards. A hypothesis closes without consuming a record. Unused prefix items are inert. Closed canonical encodings contain no initial negative parameter; fresh negative names introduced while checking may occur in the relevant continuation records.

Canonical formula encodings are

$$s = t \longmapsto [s, t], \qquad \mathsf{D}_{x,y}B \longmapsto [B], \qquad \mathsf{Q}_y B \longmapsto [B, B, B].$$

Canonical numerical encodings use scalar zero, a unary successor, and a ternary iterator. The interpretation is sorted: a list at a formula position need not have the same meaning as at a numerical position. The inherited conservative decoding of raw aliases is:

List length	Numerical position	Formula position
0	zero	$0 = 0$
1	successor of its child	D of its child
2	zero; annotations ignored	equality of its two terms
3	scalar iterator	Q of the last field
≥ 4	zero; children ignored	$0 = 0$; children ignored

The first two fields of a general ternary formula are annotations; canonical closures repeat the same matrix three times. Successfully admitted scalar padding at a formula position denotes truth.

At numerical root depth three, coordinates $0, 1, 2$ denote zero. Under p protected numerical slots, $3+2j$ and $4+2j$ name slot j, counted from the innermost slot. More generally, an admitted nonnegative scalar in $[0, 3)$ denotes zero, and one in $[3 + 2j, 5 + 2j)$ denotes protected slot j whenever $j < p$. This extends the two-integer-per-slot convention to fractional JSON numbers. Admitted negative scalar values are distinct rigid parameter names. Meaningful formula depths are $2p$, and meaningful numerical depths are $3 + 2p$.

A numerical successor adds two dummy zero slots to its child. The counter and initializer of an iterator add one dummy zero slot; its update instead adds its genuine state slot. A paired formula adds its two bound slots, and a closure adds one. Consequently the single update $d \mapsto d + 5 - \mathrm{len}(a)$ gives all required depths. Larger arities already denote constants. Traversing their ignored annotations can reject an input, but successful traversal does not change their constant interpretation. One must not infer global raw-tree idempotence from this semantic statement.

3.2 Opening, scopes, and failure sentinels

Write $\mathcal{O}_k(a; t, d)$ for the function `i` at floor k. It preserves a scalar a when $k \leq a < d$. An escaping numerical coordinate is the distinguished substitution hole; its image is checked at numerical root depth three before insertion. An escaping scalar at a formula position erases to truth. A used missing numerical image rejects. Induction on successfully traversed finite trees proves capture-free semantic opening at meaningful sorted positions. The induction treats the three meaningful arities separately; ignored annotations need no semantic induction hypothesis beyond their constant interpretation.

The internal value `0j` is the missing-image sentinel. It is not an object-language number. A successful numerical attempt to use it would require the ordered comparison `k <= 0j`, which raises `TypeError`. Thus it never enters a successfully opened term or retained hypothesis. Replacing the earlier `None` sentinel by `0j` preserves this failure behavior. Replacing it by the ordinary integer zero would silently fill missing numerical holes and violate scope validation; the supplied control then accepts an invalid closed goal containing an unallocated name.

The internal value `1j` is a deliberately unmatchable candidate. No value supplied by the standard JSON decoder equals this non-real complex value. This remains true for the decoder's additional NaN and infinity spellings. Consequently

$$m \in ([s, u],\ s \texttt{ and } 1j \texttt{ or } [t, t, t])$$

can hold only for a truthy major m. Its second alternative holds exactly when s is false-valued and $m = [t, t, t]$. The first alternative is the equality pattern. The sentinel removes the need for an extra `m and` guard; it does not remove checking of the major proof.

Using `False` for the disabled candidate would be unsound because Python identifies it with the numerical progress marker zero. The supplied mutation controls exhibit a closed false numerical equation accepted after that single bad replacement. The non-real sentinel is therefore a logical guard in this implementation, not a cosmetic choice.

3.3 Freezing an elimination before checking its major

The earlier checker first recursively proved m and then interpreted the elimination template. A successful major subproof can lower the global floor by allocating names internal to that proof. The new checker interprets the template first, at the old floor, and retains the already opened minor goal. It then proves the major before entering the continuation.

Lemma 3.1 (Frozen elimination)**.** *At a fixed floor, suppose the goal and retained context have been validated. If a matching equality or closure pattern transforms the goal using the two successful openings of one template, the later checked major justifies that transformation. Allocating names internal to the major proof does not change the decoded meaning of the already opened minor goal.*

Proof. For an equality match $m = [s, u]$, the two openings at the unchanged floor are the instances $F(u)$ and $F(s)$ of one capture-free template F. The target opening is required to equal the current raw goal. If the source opening is truthy, it becomes the minor obligation. A later successful proof of m supplies $s = u$, so transport yields the original goal. If the source opening is false-valued, the Boolean implementation keeps the original goal; this imposes the stronger unchanged obligation and remains sound.

For a closure match, s is false-valued and $m = [t, t, t]$. If its numerical image is used, successful opening forces it to denote zero; if unused, it can be replaced semantically by zero. The same opening lemma identifies the two instances as $B(u)$ and $B(0)$. After proving $\mathsf{Q}_y B$, QE justifies their use. The same false-valued source fallback is harmless.

Every meaningful negative name left in an already opened minor was admitted at the old floor. Inserted numerical images were checked at that floor too. Lowering the floor can therefore neither bind nor alter such a meaningful occurrence. Bound-coordinate identities are preserved by the sorted opening lemma. Raw contents in ignored annotations may behave differently upon later traversal, but their decoded constant meaning is unchanged. This is a semantic stability argument, not an unqualified raw-array identity claim. □

Lemma 3.2 (Folding the successful major check)**.** *Every successful invocation of* `c` *returns* `None`*. Thus, in the truthy-major branch, the expression* `c(m,g) or m*3` *proves the major and then exports exactly* `m*3`*.*

Proof. A hypothesis executes the bare `return`. Every other successful call finishes after its successful continuation without an explicit return value. Induction on the finite successful call tree gives the claim. An exception in the major propagates and prevents continuation. No unproved major can be bypassed by the Boolean expression. □

3.4 All continuation cases

Proved majors. A unary paired major $[B]$ becomes $[B, B, B]$ when multiplied by three. Opening this closure at the new fresh parameter is exactly DE's local hypothesis. Equality majors become length-six truth aliases, and closure majors become length-nine truth aliases. Other successfully admitted formula aliases have the inherited harmless constant interpretations. A mismatch in the elimination pattern retains the original goal; it still requires a proof of the major. Thus there is no "unchecked no-op" path.

Computation. When m is false-valued and u is truthy, the record supplies an arithmetic hypothesis. A false-valued s gives

$$I(0, u; v.t) = u.$$

A truthy s constructs a genuine successor counter and gives

$$I(Ss, u; v.t) = t[I(s, u; v.t)/v].$$

Raw slot conventions may introduce fresh parameters when the generated equation is opened. Its simultaneous zero instance is one of these computation theorems, so rule (1) applies. If the update uses its state, the recursive source is checked at numerical root depth three, including the update inside that source. If the state is unused, the final generated equation still validates its meaningful contents. This validation is required to prevent an escaping update coordinate from being mistaken for an extra state variable.

Paired introduction. When both m and u are false-valued, exact unary unpacking opens the paired goal at witness t, preserving its index slot. The witness is checked before the floor decreases whenever its numerical image is used. An irrelevant witness can be discarded and replaced by zero. The new goal is the successor instance at the fresh index. For the progress marker $m = 0$ an equation matrix retains its arbitrary-index instance; otherwise the retained instance is at zero. Compound matrices therefore receive only their case assumption. The exact unpacking prevents a binary equation goal from masquerading as a paired formula. The undersupplied empty case has the inherited truth interpretation.

At every non-hypothesis continuation the floor decreases by one. Earlier meaningful goals and hypotheses cannot already contain that new name. If fractional numerical aliases occur in the newly admitted interval, they are separate fresh parameters, not additional names for its integer endpoint. The fresh-discharge argument substitutes zero for all of them when required. Canonical certificates use only integer names.

3.5 Soundness and canonical completeness

Theorem 3.3 (Correspondence with the native theory)**.** *Every successful finite run of the explicit-list checker has a decoded conclusion derivable in the native theory of Section 2. Conversely, every finite canonical derivation has a finite certificate accepted by the mathematical resource-unbounded checker. The same canonical consequence relation is checked by the byte profile and by the structurally qualified generic profile.*

Proof. For soundness, induct on the finite successful checking trace. Hypothesis use is justified by semantic opening. A matched elimination uses Lemma 3.1; its major is proved by the induction hypothesis and cannot be skipped. The exported local assumption is justified by DE or truth weakening.

A computation uses its proved zero instance and the fresh-discharge rule. An introduction uses its exactly unpacked matrix, correctly scoped witness, fresh index, and restricted case or equation-progress assumption. Each continuation has the stated native rule as a derivation of its caller's original conclusion. The failure sentinels never contribute an object-language value.

For completeness, first apply Lemma 2.1 to any forbidden computation leaves. Use the canonical arity encoding and the inherited backwards serialization of the six logical rule patterns. Canonical template holes use positive escaping coordinates; all other free names are already admitted rigid parameters. Their two openings therefore give the same raw instances before and after the major subproof lowers the floor. All canonical equation, closure, and paired matrices are nonempty lists, so the false-valued source fallback does not remove a canonical inference. The sentinel guard selects exactly the intended equality or closure pattern, and the folded check proves the same major. The original finite canonical certificate is consequently accepted.

The compact list test distinguishes lists among all successfully tested JSON values: strings and numbers have different zero products, while mappings and null fail the test by exception. The generic profile has the inherited conservative treatment of additional iterable aliases; the sentinel change preserves that treatment. Its nonempty-string qualification is stated explicitly and is not needed for canonical completeness. □

The structural profile writes the goal update as an explicit conditional statement. It always adopts a matched source, even a false-valued raw alias, instead of retaining the preferred profile's stronger original-goal fallback. The direct transport or QE argument justifies this. Canonical matrices are nonempty, so this difference does not change canonical completeness. It is not a claim of identical raw acceptance across profiles.

3.6 Why raw acceptance equality is not claimed

Moving validation earlier changes how a noncanonical negative escaping coordinate is treated. Before a major subproof allocates names, it can be the template's hole. After that proof it can instead be an admitted rigid parameter. Accordingly the old and new accepted JSON languages are incomparable, although their canonical native consequence relations agree. The accompanying ordering fixtures include an input accepted only by the new checker and another accepted only by the old checker; both conclusions are true reflexive equations.

For a compact new-only example let

$$o = [0], \quad r = [0, o, 0], \quad M = [r, o], \quad C = [0, 0, o, 0].$$

The stack

$$[\, C,\ C,\ [M, r, o, [-1, o]],\ [o, o]\,]$$

is accepted by the new byte checker and rejected by its old comparator. The repeated C records prove the relevant zero computations. The template's -1 is an escaping hole before that proof allocates -1, but becomes a rigid name under the old sequencing. This example illustrates the reason for proving the new checker directly rather than asserting same-input simulation.

3.7 Finite validation and its boundary

The portable verification ran 36 inherited fixtures against the three small kernels and the readable transcription in ordinary and optimized Python: 288 process expectations, consisting of 144 accepts and 144 rejects, with no mismatches. Twenty-four additional sentinel controls rejected their false goals.

The raw-ordering checks compare the old and new byte programs on both incomparability witnesses in both execution modes. A separate eight-run mutation suite distinguishes the genuinely false equation control from the invalid closed-goal scope control. The dispatch guard was also compared on 160,000 representative JSON quadruples. Detailed receipts, payloads, and portable runners are included.

The positive cases include actual uses of equation induction, retained assumptions, nonvacuous existential witnesses, nested closures, transport, and repaired zero computations. Negative cases exercise failed majors, illegal scopes, forbidden compound progress, malformed computation descriptions, and the new sentinel guard. Mutation controls are deliberately bad programs created only in temporary test locations; they are not candidate kernels.

These checks establish reproducibility and expose specific guard failures. They do not prove the universal metatheorems or replace a proof-assistant formalization. The correctness claim rests on the written trace argument and the attributed internal interpretation proofs as well as the tests.

4 What is interpreted, and why both composites are identities

Merely implementing all standard primitive-recursive functions would not establish bi-interpretability. The interpretation must work internally at arbitrary elements of every model, preserve the complete intuitionistic language, and identify both return interpretations. The inherited construction provides these stronger conclusions.

4.1 Ordinary intuitionistic logic is reconstructed

Let $C(v,a,b) = I(v,a;u.b)$, so computation gives $C(0,a,b) = a$ and $C(Sv,a,b) = b$. On equations define

$$G_v(s=t) := (C(v,0,s) = C(v,0,t)), \qquad M_v(s=t, r=q) := (C(v,s,r) = C(v,t,q)).$$

Extend gates through paired binders. An auxiliary Q is interderivable with a D having unused witness, so it can first be eliminated from the formula grammar. Padding

$$A \dashv\vdash \mathsf{D}_{u,y} G_y A$$

aligns unequal binder depths before a selector is extended recursively. Case introduction and binder congruence prove, by external induction on finite syntax,

$$\vdash G_0 A, \quad G_{Sv} A \dashv\vdash A, \qquad M_0(A,B) \dashv\vdash A, \quad M_{Sv}(A,B) \dashv\vdash B.$$

This construction applies to arbitrary quantified native formulas; it does not claim a numerical decision procedure for their truth.

Put $p(z) = I(z,0;u.C(u,1,0))$. The fixed computations $p(1) = 1$ and $p(2) = 0$ suffice for the logical macros

$$\begin{aligned}
A \Rightarrow_* B &= \mathsf{D}_{u,z} M_z(A,B),\\
A \wedge_* B &= \mathsf{D}_{u,z} G_z M_{p(z)}(A,B),\\
\exists_* x\, A(x) &= \mathsf{D}_{x,y} G_y A(x),\\
A \vee_* B &= \exists_* x\, M_x(A,B),\\
\forall_* x\, A(x) &= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z),\\
\bot_* &= (0=1).
\end{aligned}$$

Newly introduced variables are fresh and u is unused. Selector endpoints give implication introduction and elimination; gates give the existential rules. Numerical cases give disjunction elimination. The instances at 1 and 2 give conjunction projections. Derived numerical coverage handles arbitrary targets in universal elimination. Under $0 = 1$, transport through C gives every equation, and external formula induction with case introduction gives explosion.

The exact native formula returns are

$$\mathsf{Q}_y B \dashv\vdash B(0) \Rightarrow_* \forall_* y B(y),$$
$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x,0) \Rightarrow_* \forall_* y B(x,y)\big).$$

Forwards, eliminate the paired formula, assume its base, and use the opened closure at a fresh arbitrary index. Backwards, eliminate the displayed existential, introduce D by case at that witness, and use implication and universal elimination to supply the successor instance. These are return equivalences for arbitrary native matrices, not just for a chosen image of ordinary formulas. The Q return follows by the unused-witness equivalence above, or directly by closure elimination and universal introduction.

4.2 Arithmetic reconstruction in the required order

The ordinary realization T_I has intuitionistic first-order equality logic, the scalar computation schemas, numerical coverage $\forall x(x = 0 \vee \exists y\, x = Sy)$, the explicit separation bridge $(0 = S0) \to \bot$, and contextual literal-equation induction. The bridge must be present: positive numerical equations alone admit a singleton structure. Native falsity maps to the designated contradictory equation and ordinary falsity maps back to it.

The appendices establish the following dependency order internally. First, scalar iteration and equation induction construct a predecessor term P with $P0 = 0$ and $P(Sn) = n$. No pairing or equality test is assumed in that bootstrap. Next, truncated subtraction and a characteristic of actual equality are constructed, and their reflection is proved on the whole domain. This compiles each quantifier-free formula to an equivalent equation, yielding quantifier-free induction. Only then are pairing and its projections derived by bounded scalar iteration. Thus none of these steps silently uses the general primitive-recursion theorem it is supposed to establish.

For a fixed primitive-recursive description

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections, and let the earlier descriptions have scalar translations g^*, h^*. Set

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big),$$
$$f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$, with passive parameters fixed. The projection equations then establish the target recursion equations. This is an external recursion on each finite function description. It is not a uniformly total numerical evaluator.

Theorem 4.1 (Full bi-interpretability and equiconsistency)**.** *The canonical presentation checked by the new kernels is bi-interpretable and equiconsistent with full first-order iPRA. Both interpretations use the entire numerical domain, interpreted equality is actual equality, and the numerical composite isomorphisms have graph $x = y$.*

Proof. Interpret a fixed scalar iterator in iPRA by ordinary primitive recursion whose update ignores the old counter. Interpret Q, D by their displayed ordinary meanings. Coverage proves case introduction, and equation induction proves progress introduction under an assumed base. The other rules have their ordinary logical meanings.

In the reverse direction use the logical macros and the scalar compiler above. The internal predecessor, reflection, and quantifier-free-induction proofs validate all target arithmetic axioms. For any fixed iterator, equation induction compares two operations satisfying its same seed and update equations and proves uniqueness at arbitrary internal counters. The analogous argument applies to ordinary primitive recursion. External induction on finite descriptions therefore identifies every numerical double translation with the original operation. The identity graph is internally a total bijection preserving the operations.

The displayed Q and D return equivalences, together with the ordinary returns of the six logical macros, prove both formula composites by external structural induction. Falsity is preserved using the explicit ordinary separation bridge. Finite proof translations thus preserve contradictions in both directions. No assertion that either theory proves its own consistency is made.

Lemma 2.1 preserves the native consequence relation under the computation restrictions, and Theorem 3.3 identifies that canonical consequence relation with the new checker. Hence the implementation changes preserve all these interpretation properties. □

5 Strict dependence in the positive-iteration term algebra

Status and scope. The current checker-and-barriers draft proves a zero-profile obstruction for positive iteration. The certified-zero draft gives the stronger, whole-theory obstruction of a two-element model for the proposed eager arithmetic, even with induction on every formula. The results below concern a different question: the exact functions denoted by the positive-iteration terms in the *standard natural numbers*. They strengthen the standard-function analysis, but do not remove that finite-model obstruction or establish a smaller iPRA presentation. A search of the supplied TeX corpus found no statement of the strict-dependence classification below; no literature-wide priority claim is made.

The term algebra. Let terms be built from variables, zero, successor, and

$$F(n, a; u.h) = h_{\vec{x}}^{n+1}(a), \qquad h_{\vec{x}}(b) = h(b, \vec{x}).$$

Only u is bound in the update h. All passive parameters are fixed during iteration, the counter is not passed to the update, and each description is finite earlier-stage syntax. Arbitrary nesting and capture-free substitution are allowed. Equivalently,

$$F(0, a; u.h) = h[a/u], \qquad F(Sn, a; u.h) = h[F(n, a; u.h)/u].$$

The following assertions are about this standard interpretation, not about every model of those equations.

For a function $f : \mathbb{N}^r \to \mathbb{N}$, a coordinate j is *essential* if changing that coordinate can change the value, with the other coordinates fixed. Write $U(f)$ for its set of essential coordinates. Strict increase in coordinate j means strict increase for *every* fixed choice of the other coordinates, not merely for some such choice.

Theorem 5.1 (Constant–projection–growth classification)**.** *Every fixed positive-iteration term $t(x_1, \ldots, x_r)$ belongs to exactly one of the following three classes.*

1. *It is a constant function.*
2. *It is one coordinate projection x_j.*
3. *Its essential-coordinate set U is nonempty; it is independent of the remaining coordinates; it is strictly increasing in each coordinate in U; and*

$$t(\vec{x}) \geq x_j + 1 \qquad (j \in U).$$

In particular every term is coordinatewise nondecreasing. Its class and its exact essential-coordinate set can be calculated by structural recursion on its finite syntax. Calculating the class does not require evaluating a constant term or uniformly evaluating arbitrary primitive recursive descriptions.

Proof. We prove the classification, monotonicity, and the asserted exact dependencies together. Zero is constant and a variable is its projection. Successor preserves constancy for a constant argument. For a nonconstant argument it preserves its exact dependencies and strict increases, and raises its lower bound from at least x_j to at least $x_j + 1$.

Consider $F(n, a; u.h)$. Apply the induction hypothesis to n, to a, and to h, regarding its state u as an additional independent coordinate. There are four immediate cases.

1. If h is constant, the result is that constant: at least one update is performed.
2. If h is the state projection u, the result is the seed a.
3. If h is a passive-coordinate projection, the result is that projection, independently of the seed and counter.
4. If h has the third class but does not depend on u, the result is $h(\vec{x})$, with exactly its already classified dependencies.

The remaining case is that h has the third class and depends on u. For every fixed passive assignment, therefore,

$$h(b, \vec{x}) \geq b + 1,$$

and h is strictly increasing in b. Let U_n, U_a be the essential sets of the counter and seed, and let V be the passive essential set of h. Define

$$U = U_n \cup U_a \cup V.$$

The resulting function is independent of coordinates outside U. If U is empty it is constant. Suppose U is nonempty.

First, the result is monotone in every coordinate. For two ordered parameter assignments $\rho \leq \sigma$, the seed and counter values are ordered, and the update under ρ is pointwise at most the update under σ. Induction over any common number of updates preserves the ordered states. Any additional updates at σ can only increase its state, because each update raises the state by at least one.

Next, every coordinate in U gives a strict increase. An increase of an essential seed coordinate strictly increases the initial state, and strict monotonicity of the update in its state preserves that difference. An increase of an essential passive update coordinate strictly increases the state at the first update, which exists even at counter zero; later updates preserve the difference. An increase of an essential counter coordinate increases the number of updates, and every extra update raises the state by at least one. If a coordinate occurs in more than one of these roles, all its other effects are nondecreasing, so the same strict comparison remains valid. Thus U is precisely the essential-coordinate set.

Finally, writing N, A for the counter and seed values, the result is at least $A + N + 1$. If $j \in U_a$, the induction hypothesis gives $A \geq x_j$, so the result is at least $x_j + 1$. If $j \in U_n$, it gives $N \geq x_j$, with the same conclusion. If $j \in V$, the last application of h is at least $x_j + 1$ by the induction hypothesis on h. This proves the required dominance for every $j \in U$.

These cases also constitute a finite classification algorithm. Constants and projections have their explicit tags. In the last case one forms the union $U_n \cup U_a \cup V$ and tests whether it is empty. No semantic search or evaluation of all assignments is involved. The three outcomes are disjoint: a projection fails the strict additive lower bound at that coordinate, and a constant has no essential coordinate. □

Corollary 5.2 (No bounded nonconstant numerical term)**.** *Every nonconstant positive-iteration term is unbounded in each of its essential coordinates with all other coordinates fixed. Consequently no term denotes a nonconstant function with finite range. In particular there is no Boolean-valued zero-test term, and no predecessor term.*

Proof. A projection is unbounded in its sole essential coordinate. In the third class the lower bound $t(\vec{x}) \geq x_j + 1$ proves unboundedness. Predecessor is nonconstant but is not strictly increasing from 0 to 1. □

5.1 An obstruction for tuple domains with actual equality

The classification also obstructs moving to finite tuples if interpreted equality remains actual coordinatewise equality. Quotient equality is a material exception, as the next subsection demonstrates explicitly.

Theorem 5.3 (No nontrivial term-defined unital magma on a tuple domain)**.** *Let d be finite, let $\varnothing \neq D \subseteq \mathbb{N}^d$, and let $e \in D$. Suppose $f : D \times D \to D$ is given by a fixed tuple of positive-iteration terms, with any fixed numerical parameters, and*

$$f(e, z) = z = f(z, e) \qquad (z \in D)$$

in actual coordinatewise equality. Then D has exactly one element. No definability assumption on D is needed.

Proof. Fixed numerical parameters can be replaced by their numeral terms, so Theorem 5.1 applies to every coordinate function $f_i(x_1, \ldots, x_d, y_1, \ldots, y_d)$. Let J be the coordinates that are nonconstant on D. If $i \in J$, f_i must depend on at least one coordinate from each input tuple: otherwise one of $f_i(e, z)$ and $f_i(z, e)$ would be constant on D. In particular f_i is neither constant nor a projection, and has the third class.

There is an essential left-input coordinate $j_i \in J$. Indeed, if every essential left-input coordinate were constant on D, then $f_i(z, e)$ would be constant there, contrary to $i \in J$. The dominance conclusion and the right unit law yield

$$z_i \geq z_{j_i} + 1 \qquad (z \in D).$$

If J were nonempty, the map $i \mapsto j_i$ on the finite set J would have a directed cycle. Summing the displayed inequalities around that cycle gives $z_i \geq z_i + \ell$ for its positive length ℓ, an impossibility. Thus J is empty. Every coordinate is constant on D, so the nonempty set D is a singleton. The case $d = 0$ is immediate. □

5.2 The same algebra does have an injective pairing term

The preceding theorem is not an almost-unary majorization result. Positive iteration combines two unbounded inputs, and even defines an injective numerical pairing in the standard model. Put

$$q(a,b) := F(a,b;u.Su) = a+b+1, \qquad d(a) := q(a,a) = 2a+1,$$

and define

$$\pi(a,b) := F(a,S(d(b));u.d(u)).$$

Iteration of $u \mapsto 2u+1$ gives the exact identity

$$\pi(a,b)+1 = 2^{a+1}(2b+3).$$

The factor $2b+3$ is odd. Uniqueness of the power of two dividing a positive integer therefore recovers a from $\pi(a,b)$; the odd part then recovers b. Thus π is injective on $\mathbb{N}^2$.

This is not a pairing compiler for arbitrary primitive recursion. A numerical decoder is not supplied by the displayed term, and the classification prevents the usual coordinate projections of this pairing from being positive-iteration unary terms. For example, either projection is nonmonotone on suitable values in the range of π, while a nonconstant unary term is strictly increasing on all of $\mathbb{N}$. For the first projection use $\pi(1,0) = 11 < 13 = \pi(0,2)$; for the second use $\pi(0,1) = 9 < 11 = \pi(1,0)$.

5.3 Why quotient equality is a substantive escape

There is an explicit standard-model quotient with term-defined ring operations. This illustrates the limits of Theorem 5.3, not a bi-interpretation of the whole eager theory. In addition to q, put

$$m(a,b) := F(a,0;u.q(u,b)) = (a+1)(b+1).$$

On $\mathbb{N}^2$ define

$$(a,b)\ E\ (c,d) \quad \Longleftrightarrow \quad q(a,d) = q(c,b).$$

This is exactly equality of the integer differences $a-b$ and $c-d$. The map $(a,b) \mapsto a-b$ identifies $\mathbb{N}^2/E$ with $\mathbb{Z}$. Zero is represented by $(0,0)$, the multiplicative unit by $(1,0)$, and negation swaps coordinates. Addition and multiplication are given by the following *tuples of positive-iteration terms*:

$$\begin{aligned}(a,b) \oplus (c,d) &:= (q(a,c), q(b,d)),\\ (a,b) \otimes (c,d) &:= (q(m(a,c),m(b,d)),\ q(m(a,d),m(b,c))).\end{aligned}$$

For addition, the two extra ones cancel. For multiplication, expanding the difference of the two coordinates gives

$$(a+1)(c+1)+(b+1)(d+1)-(a+1)(d+1)-(b+1)(c+1) = (a-b)(c-d).$$

Thus both operations respect E and induce the usual integer ring. The nonnegative-difference domain also has the first-order description

$$a = b \quad \vee \quad \exists r\,(q(b,r) = a).$$

These are standard-algebra identities. No claim is made here that the eager axioms prove the required quotient properties, transfer induction to graph-defined arithmetic, or supply closure under arbitrary primitive recursion.

In fact the inherited two-element eager model has $Sx = 1$ and interprets $F(n,a;h)$ as $h(a)$. There $q(a,b) = 1$ for all inputs, so the same formula E identifies every pair and the displayed quotient has one element. This is a concrete reason why a successful construction in the standard algebra is insufficient for interpretation of iPRA in the whole proposed eager theory. Adding successor injectivity excludes that particular model, but proving a new interpretation after such an addition would be further work.

Verification boundary. The companion Python program checks a deterministic finite sample of the classification algorithm, and the explicit pairing and quotient formulas. It does not verify the structural induction proof in a proof assistant. The proofs above establish the unbounded standard-function statements; the tests check their transcription on the reported finite inputs.

6 Larger redesigns examined, and the remaining target

The search did not stop at the final source refactoring. Several larger changes were examined, but none produced a smaller justified foundation. The distinctions below are useful because each tempting deletion moves an essential obligation somewhere else.

6.1 Synthesizing the conclusion

A proof checker can infer its conclusion instead of checking an explicitly provided goal at each recursive call. Hypothesis records then carry context indices, and major formulas and equality endpoints need not be repeated in certificates. Two standalone experimental implementations are supplied under `experiments/synthesis/`.

The transparent experiment measures 951 normalized bytes, 330 tokens, and 390 AST nodes; a shared-binder version measures 777 bytes, 334 tokens, and 392 nodes. Both include advertised-goal comparison. Seventy-six focused process expectations passed. These remain experimental programs, not newly established complete replacements for the small kernels.

The source cost is informative: inference requires explicit hypothesis dispatch, equality or closure minor-instance comparison, comparison of the inferred introduction premise with the required successor formula, and validation that an inferred elimination conclusion contains no escaping eigenparameter. Reducing certificate redundancy did not reduce the trusted checker. The two resources should be measured separately.

6.2 Making zero computation definitional

Identifying $I(0, a; h)$ with a in syntax is mathematically legitimate. It does not make the implementation work free. In the current codec an iterator seed is placed under a dummy zero slot. For example, in

$$I(n, a; u.I(0, u; v.0)),$$

the occurrence of u inside the inner zero iterator's seed has crossed that extra slot. Removing the inner iterator must lower its coordinate from five to three in the surrounding update body. Simply returning the raw seed does not preserve scope. A correct normalizer has to implement this lowering, validate the resulting term, and preserve equality transport and retained-hypothesis lookup modulo the conversion. The examined normalization machinery was larger than the computation dispatch it would remove.

6.3 Using the remaining certificate length as a scope clock

This proposal could remove a mutable global counter, but consuming a record advances such a clock before its witness has been checked. The would-be fresh introduction parameter can then already be used in the witness. The logical failure has a short explicit example. Let

$$E(x, y) := (C(y, 0, x) = y), \qquad C(y, a, b) = I(y, a; u.b).$$

The zero instance is always true, whereas $E(x, Sz)$ says $x = Sz$. Consequently $\mathsf{D}_{x,y}E(x, y)$ is false in standard arithmetic: one fixed x cannot equal both 1 and 2. An illegal case introduction choosing the witness $t = Sz$ nevertheless has the immediate computation premise $C(Sz, 0, Sz) = Sz$. The witness-freshness condition excludes exactly this mistake. A corrected certificate clock must reserve a separate allocation phase or implement an equivalent guard; the examined versions did not save source.

6.4 Deleting major proofs or arithmetic initialization

The current drafts already supply precise obstructions to several other proposals. Requiring every elimination major to be a retained hypothesis loses even the closed theorem $\mathsf{Q}_y(0 = 0)$ in the specified unchanged calculus [8]. Merely removing Q from public goals does not restore equation induction: a closed equation derivation then lacks the route through an introduced progress major. A focused replacement would need a charged induction rule and a corresponding normalization proof.

Likewise, tying a recursor's seed to its update or eagerly applying the update at zero is not automatically a scalar basis for iPRA. The certified-zero report gives a two-element model for the eager equations, even with its stated coverage, separation, and induction conditions [2]. Other current reports establish broader carefully scoped term and interpretation obstructions [8, 9]. These results are not blanket impossibility theorems about every possible changed domain or relational interpretation.

6.5 Assessment

The checked improvement is nineteen normalized bytes in the byte profile, with smaller token and AST profiles provided separately. The explicit-list program is itself smaller in bytes than the old compact byte leader. The ablation separates the proof-order change from the small Python sentinel choices, so the numerical saving is not presented as a new arithmetic axiom reduction.

No significant new axiomatization, absolute optimum, or single program dominating every old coordinate has been established. The remaining mathematical objective is a presentation with fewer genuinely necessary checking mechanisms, together with internal proofs of both interpretation composites. The present package supplies a reproducible smaller checkpoint, the complete inherited foundation it depends on, and precise records of why the tested larger shortcuts were not accepted.

A Attribution and the complete inherited mathematical bridge

The five following appendices are copied without textual modification from the reference material of the certified-zero archive in the pinned snapshot. That archive attributes them in turn to the arity-calculus, Choice-DI, scalar predecessor, equality-reflection, and progress-closure developments. Their exact hashes and provenance are in `evidence/source_snapshot.json`; the repository license is included. Any statement described as "new" within one of these inherited appendices belongs to its original report, not to the present investigation. The appendices are included to supply the entire numerical and logical bridge, rather than outsourcing the difficult predecessor and induction arguments to an unavailable document.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse

direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [11] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{2}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{3}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{4}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{5}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{6}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (6) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{7}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (6); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{8}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (7), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{9}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{10}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (10). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (11) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (12) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (13)$$

The base is (12). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (14)$$

The first uses the comparator recurrence. The second uses (13) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (15)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (16)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (15) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [12]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{17}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{18}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{19}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{20}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{21}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (21) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{22}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{23}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{24}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (17). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (23); symmetry and (20) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{25}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{26}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{27}$$

To prove the range assertion, split (19). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (18) also give, for all u, v,

$$C(u,v,v) = v. \tag{28}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{29}$$

If $u = 1 = S0$, (24) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{30}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{31}$$

Using these equations and (21), we now derive

$$0 + y = y, \tag{32}$$
$$(Sx) + y = S(x + y), \tag{33}$$
$$d(Sx, Sy) = d(x, y), \tag{34}$$
$$d(Sa, a) = 1. \tag{35}$$

Each induction below has a single equation as its induction formula.

For (32), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (33), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (33) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{36}$$

Indeed, both sides are $S(y + r)$.

For (34), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (35). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (34) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{37}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (32) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (28).

For the step, assume (37) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (37) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (36) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (37). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{38}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (37). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (35) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}(K(d(Sa,b))). \tag{39}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{40}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{41}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (26), which is equivalent to $d(Sa,b) = 1$ by (29), and then to $a = b$ by (38). The range assertion is (25) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b)=0$ branch, (40) gives $a=b$. In the $e(a,b)=1$ branch, an assumption $a=b$ would give $e(a,b)=0$ and therefore $0=1$, contradicting separation. Hence

$$\forall a\,\forall b\,\big(a=b\vee(a=b\to\bot)\big) \tag{42}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t}=e(s,t), \qquad b_\bot=1, \tag{43}$$
$$b_{A\wedge D}=C(b_A,b_D,1), \tag{44}$$
$$b_{A\vee D}=C(b_A,0,b_D), \tag{45}$$
$$b_{A\to D}=C(b_A,b_D,0). \tag{46}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A=0\vee b_A=1\big)\quad\text{and}\quad\big(b_A=0\longleftrightarrow A\big). \tag{47}$$

Here are the full connective checks. In every case $b_A=1$ implies $\neg A$: if A held, the already established implication $A\to b_A=0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (40)–(41). For falsity the range assertion is $1=0\vee 1=1$, with its right disjunct proved by reflexivity. Also $1=0\to\bot$ follows from separation and symmetry, while $\bot\to 1=0$ follows by explosion. Thus $1=0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A=0\vee b_A=1$. In the zero branch, A holds and $b_{A\wedge D}=b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A\wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D}=1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A\wedge D$; conversely an assumption $A\wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D}=0$.

Disjunction. In the branch $b_A=0$, the formula A holds and $b_{A\vee D}=0$. The range is immediate, and both $b_{A\vee D}=0$ and $A\vee D$ hold, proving their equivalence. In the branch $b_A=1$, we have $\neg A$ and $b_{A\vee D}=b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A\vee D$. Conversely, from $A\vee D$, the A alternative contradicts $\neg A$ and yields $b_D=0$ by explosion, while the D alternative yields $b_D=0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (47) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (47), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [13]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt, current draft collection*, commit `76a2651601d17b34d46753f189ea026a21ce8d29`. https://github.com/VladimirReshetnikov/ProveIt/tree/76a2651601d17b34d46753f189ea026a21ce8d29/docs/drafts.

[2] *A Certified Zero Removes a Computation Opcode*, `ipra_certified_zero_2026-11-10.zip` in [1]. Direct source of the comparison checker, initializer dispatch, three-node computation repair, test producer, and inherited appendices.

[3] *An Arity Calculus for iPRA*, `ipra_arity_calculus_2026-11-10.zip` in [1]. Source of the unary/triplicate representation and affine scope traversal.

[4] *Uniform Predecessor Certificates for Intuitionistic PRA*, `ipra_uniform_predecessor_2026-11-10.zip` in [1].

[5] *Constructed Computation*, `ipra_constructed_computation_2026-11-10.zip` in [1].

[6] *Numerical Axioms as Local Assumptions*, inherited in the reference material of [3, 5].

[7] *A One-State Recursor for iPRA*, `ipra_transient_seed_2026-11-10.zip` in [1].

[8] *iPRA: Adaptive Seeds, Finite Quotients, and the Current Checker Frontier*, `ipra_adaptive_seed_2026-11-10.zip` in [1].

[9] *Local Proof Scopes and Interpretation Obstructions for iPRA*, `ipra_local_scopes_2026-11-10.zip` in [1].

[10] Saeed Salehi. *Decidable Formulas of Intuitionistic Primitive Recursive Arithmetic*. Reports on Mathematical Logic **36** (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.

[11] Inherited scalar predecessor development, retained with attribution in `reference/scalar_predecessor.tex` of [2].

[12] Inherited forward-left equality-reflection development, retained in `reference/equality_reflection.tex` of [2].

[13] Inherited progress-closure pairing and primitive-recursion compiler, retained in `reference/primitive_recursion.tex` of [2].